

K.S.R. COLLEGE OF ENGINEERING

An Autonomous Institution

KSR KALVI NAGAR

TIRUCHENGODE – 637215

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RECORD NOTEBOOK

24ITP56& NETWORKS LABORATORY



NAME :.....

REGISTER NUMBER :.....

YEAR/SEM :.....

K.S.R. COLLEGE OF ENGINEERING

An Autonomous Institution

TIRUCHENGODE – 637 215

CERTIFICATE

Register No.....

Certified that this is a bonafide record of work done..... by
Selvan/Selvi of the.....Semester
.....Branch during the year.....in the
.....Laboratory.

Staff in-charge

Head of the Department

Submitted for the Practical Examination on.....

Internal Examiner

External Examiner

CONTENTS

[illegible]

CONTENTS

[illegible]

Vision of the Institution

- IV** To become a globally renowned institution in Engineering and Management, committed to providing holistic education that fosters research, innovation and sustainable development.

Mission of the Institution

- IM 1** Deliver value-based quality education through modern pedagogy and experiential learning.
- IM 2** Enrich Engineering and Managerial Skills through cutting-edge laboratories to meet evolving global demands.
- IM3** Empower research and innovation by integrating collaboration, social responsibility, and commitment to sustainable development.

Vision of the Department / Programme: (Computer Science and Design)

- DV** To produce professionals for designing technology with ethical values, ingenious attitude and team spirit required for the continual development of the society and the nation.

Mission of the Department / Programme: (Computer Science and Design)

- DM 1** To bestow academic environment for the development of skilled professionals qualified with knowledge, skills, values, and ethics, thereby take a role in the field of computer science and design
- DM 2** Imbibing holistic, creative learning and ethical attitude for embracing global challenges and leadership qualities in the field of computer science and design.
- DM 3** To influence graduates with the skills to become self-employed entrepreneurs and future leaders.

Programme Educational Objectives (PEOs): (Computer Science and Design)

The graduates of the programme will be able to

- PEO 1** To develop the ability to think critically, analyse and make innovative design for offering techno-commercially feasible solutions.
- PEO 2** To apply current tools and technologies to contribute for industries, public sectors, research organization for solving time critical problems.
- PEO 3** To impart the knowledge of inventive design skills and lifelong learning to succeed in their professional challenges.

24ITP56	NETWORKS LABORATORY	Category	L	T	P	SL	C
		PCC	0	0	45	0	1.5

Common to B.Tech IT, AIDS, B.E CSE, CSE (IOT), CSE (CS)

PREREQUISITE:

Students should have basic knowledge of networks, programming concepts, and operating systems.

OBJECTIVES:

To understand and implement fundamental computer networking concepts, network topologies, devices, and LAN configuration; apply and implement data communication, routing techniques and develop TCP/UDP based client server applications; and simulate, analyze, and evaluate network protocols and performance using network simulation tools such as NS2/NS3.

List of Experiments:

- 1 Study Various Network Topologies and analyze the Functionality of Network Devices.
- 2 Develop and Implement a Program for Error Detection Using the Cyclic Redundancy Check (CRC) Technique.
- 3 Design and Develop a Program for Flow Control Mechanisms Using Sliding Window Protocol.
- 4 Create a Program to Implement Stop-and-Wait ARQ Protocol for Reliable Data Transmission.
- 5 Write a Program to Implement Distance Vector Routing Using the RIP Protocol.
- 6 Construct a Program to Implement Link State Routing Using the OSPF Protocol.
- 7 Develop a program to Implement Client Server Communication Using TCP for Date and Time Services.
- 8 Build a program to implement a simple TFTP client-server application using Read Request (RRQ) and Write Request (WRQ) operations.
- 9 Construct a program to Implement Client Server Communication Using UDP Sockets
- 10 Create a program to implement and Analyze Domain Name Resolution Using the Domain Name System (DNS).
- 11 Develop a program to implement client-server application to demonstrate HTTP request and response message exchange
- 12 Study Network Simulation Using the NS2/NS3 Network Simulator.

L=0, T=0, P=45, SL=0, TOTAL: 45 PERIODS

COURSE OUTCOMES:

At the end of the course, the students will be able to:

COs	Course Outcome	Experiments	Cognitive Level
CO1	Study and analyze computer network topologies, network devices, and Local Area Networks (LANs)	1	Analyze
CO2	Implement error detection, reliable data transmission techniques, and routing algorithms to ensure efficient communication in computer networks	2, 3, 4	Apply
CO3	Apply the concepts of the network layer and analyze Distance Vector Routing and Link State Routing protocols in computer networks.	5, 6	Apply
CO4	Develop and test client server applications using TCP and UDP socket programming for network based services	7, 8, 9	Analyze
CO5	Apply the concepts of application layer protocols to analyze web communication, file transfer, email services, and domain name resolution and gain the knowledge about simulation tool used in computer networks.	10,11,12	Apply

REFERENCES:

- 1 James F. Kurose and Keith W. Ross, Computer Networking: A Top-Down Approach, Pearson, Seventh Edition, 2017
- 2 Behrouz A. Forouzan, Data Communications and Networking, McGraw-Hill Education, Fifth Edition, 2013

LIST OF EQUIPMENTS (For a Batch of 30 Students)

S. No	Name of the Equipment's	Qty.
1.	30 Systems / Work Stations for a batch with LAN Connectivity and network simulation tools such as NS2 / NS3	30 Nos.
2.	Programming Languages: C / Python / Java	

Mapping of COs with POs and PSOs

COs/ POs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO 1	PSO2
CO1	3	2	1	1	2	-	-	-	-	-	1	3	2
CO2	3	3	2	2	2	-	-	-	-	-	1	2	3
CO3	3	3	2	2	3	-	-	-	-	-	1	3	2
CO4	3	2	1	2	2	-	-	-	-	-	1	2	3
CO5	3	2	1	3	2	-	-	-	-	-	1	3	2

1-low, 2-medium, 3-high

EX.NO.: 1	STUDY VARIOUS NETWORK TOPOLOGIES AND ANALYZE THE FUNCTIONALITY OF NETWORK DEVICES
DATE:	

AIM

To study different network topologies (Star, Bus, Ring, Mesh) and to write a Java program that builds a topology and simulates the functionality of network devices such as Hub, Switch, and Router.

ALGORITHM

1. Start the program.
2. Read the number of nodes (n) and the topology choice: 1-Star, 2-Bus, 3-Ring, 4-Mesh.
3. Build the adjacency matrix: Star links node 0 to all nodes; Bus and Mesh link every node to every other node; Ring links each node to the next node, and the last node back to the first.
4. Display the adjacency matrix and count the total number of physical links.
5. Read the device choice (1-Hub, 2-Switch, 3-Router), the source node, and the destination node.
6. If the device is Hub, broadcast the frame to every node except the source.
7. If the device is Switch, forward the frame only to the destination node.
8. If the device is Router, route the packet toward the network of the destination node.
9. Display how the chosen device handled the frame.
10. Stop the program.

PROGRAM

```
import java.util.Scanner;

/*
 * Experiment 1: Study Various Network Topologies and analyze the
 * Functionality of Network Devices.
 * This program lets the user:
 * 1. Build and display a network topology (STAR, BUS, RING, MESH) using an adjacency matrix.
 * 2. Simulate how different network devices (HUB, SWITCH, ROUTER) handle an incoming data
 * frame, so their functional difference can be observed.
 */
public class NetworkTopologySimulator {

    // Builds and prints the adjacency matrix for the chosen
    topology static void buildTopology(int n, int choice) {
        int[][] adj = new int[n][n];

        if (choice == 1) { // STAR : node 0 is the central
            hub/switch for (int i = 1; i < n; i++) {
                adj[0][i] = 1;
            }
        }
    }
}
```

```

        adj[i][0] = 1;
    }
} else if (choice == 2) { // BUS : all nodes share one common
    line for (int i = 0; i < n; i++)
        for (int j = 0; j < n;
            j++) if (i != j)
                adj[i][j] = 1;
} else if (choice == 3) {
    // RING : each node connects to next, last connects to
    first for (int i = 0; i < n; i++) {
        int next = (i + 1) % n;
        adj[i][next] = 1;
        adj[next][i] = 1;
    }
} else if (choice == 4) { // MESH : every node connects to every other
    node for (int i = 0; i < n; i++)
        for (int j = 0; j < n;
            j++) if (i != j)
                adj[i][j] = 1;
}
System.out.println("\nAdjacency Matrix of the Topology:");
System.out.print("  ");
for (int i = 0; i < n; i++)
    System.out.print("N" + i + " ");
System.out.println();
for (int i = 0; i < n; i++) {
    System.out.print("N" + i + " :
"); for (int j = 0; j < n; j++)
        System.out.print(adj[i][j] + " ");
    System.out.println();
}
int totalLinks = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n;
        j++)
        totalLinks += adj[i][j];
System.out.println("Total number of physical links = " + (totalLinks / 2));
}

// Simulates how a data frame sent by a source node is handled
// by different network devices
static void simulateDevice(int deviceChoice, int source, int dest, int n) {
System.out.println("\nNode " + source + " sends a frame intended forNode " + dest);
    if (deviceChoice == 1) {
        System.out.println("Device = HUB (Layer 1 device)");
        System.out.println("Functionality: A hub simply repeats/broadcasts the "+ "incoming
signal to ALL other ports without "+ "checkingaddresses.");
        for (int i = 0; i < n; i++)
            if (i != source)

```

```

        System.out.println(" -> Frame broadcast to Node " + i);

    } else if (deviceChoice == 2) {
        System.out.println("Device = SWITCH (Layer 2 device)");
        System.out.println("Functionality: A switch learns MAC addresses and "+ "forwards the
        frame ONLY to the destination port.");
        System.out.println(" -> Frame forwarded directly to Node "
        + dest + " only");
    } else if (deviceChoice == 3) {
        System.out.println("Device      =      ROUTER      (Layer      3      device)");
        System.out.println("Functionality: A router reads the destination IP "+ "address and
        forwards the packet along the best path "+ "between different networks.");
        System.out.println(" -> Packet routed toward network "+ "containing Node " + dest);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("---- Network Topology Simulator      ");
    System.out.print("Enter number of nodes: ");
    int n = sc.nextInt();

    System.out.println("Choose Topology: 1.Star  2.Bus  3.Ring
    4.Mesh"); int tChoice = sc.nextInt();
    buildTopology(n, tChoice);

    System.out.println("\n---- Network Device Functionality  ");
    System.out.println("Choose Device:  1.Hub    2.Switch
    3.Router"); int dChoice = sc.nextInt();

    System.out.print("Enter source node number: ");
    int src = sc.nextInt();
    System.out.print("Enter destination node number: "); int
    dst = sc.nextInt();

    simulateDevice(dChoice, src, dst, n);
    sc.close();
}
}

```

SAMPLE OUTPUT

```
---- Network Topology
Simulator ----Enter number
of nodes: 4
Choose Topology: 1.Star 2.Bus 3.Ring
4.Mesh 1

Adjacency Matrix of the
Topology: N0 N1 N2
N3
N0 : 0  1  1  1
N1 : 1  0  0  0
N2 : 1  0  0  0
N3 : 1  0  0  0
Total number of physical links = 3

---- Network Device Functionality
----Choose Device: 1.Hub
2.Switch 3.Router 1
Enter source node
number: 0 Enter
destination node number:
2
```

PRE LAB QUESTIONS

1. Define network topology and name its four basic types.
2. Differentiate between a Hub and a Switch.
3. Identify the OSI layer at which a Router operates.
4. Justify why a Mesh topology is considered the most reliable but the most expensive.
5. Distinguish between a Bridge and a Switch.

POST LAB QUESTIONS

1. Determine which topology requires the maximum number of physical links for n nodes, and state how many links it needs.
2. Why a Hub causes more collisions on a network compared to a Switch?
3. Modify the program to simulate a Bridge, and describe the functional difference between a Bridge and a Router.
4. Analyze the sample output and account for the Hub sending the frame to Node 1 and Node 3 even though it was meant only for Node 2.
5. List one advantage and one disadvantage of the Star topology

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT :

Thus a Java program was developed to build different network topologies and simulate the functionality of network devices such as Hub, Switch, and Router. The program was executed successfully and the output was verified for a Star topology with a Hub broadcasting the frame to all connected nodes.

EX.NO.: 2	ERROR DETECTION USING CYCLIC REDUNDANCY CHECK (CRC)
DATE:	

AIM

To write a Java program to implement error detection in transmitted data using the Cyclic Redundancy Check (CRC) technique.

ALGORITHM

1. Start the program.
2. Read the data bits and the generator (key) bits from the user.
3. Let n be the length of the key. Append $(n - 1)$ zero bits to the data to form the appended data.
4. Perform binary (XOR) division of the appended data by the key to obtain the remainder (CRC bits).
5. Form the codeword by appending the CRC bits to the original data. This is transmitted by the sender.
6. At the receiver side, read the received codeword (the user may enter a modified codeword to simulate a transmission error).
7. Divide the received codeword by the same key using XOR division.
8. If the remainder is all zeros, declare 'No error detected'.
9. Otherwise, declare 'Error detected in received data'.
10. Stop the program.

PROGRAM

```
import java.util.Scanner;

/*
 * Experiment 2: Develop and Implement a Program for Error Detection
 * Using the Cyclic Redundancy Check (CRC) Technique.
 * Sender side : appends CRC (remainder) to the message and sends the codeword.
 * Receiver side: divides the received codeword by the same generator polynomial.
 * If the remainder is all zeros, the data is accepted as error-free, otherwise an error is detected.
 */
public class CRC {

    // Performs binary (XOR based) division and returns the remainder
    static String xorDivision(String data, String key) {
        int n = key.length();
        char[] rem = data.substring(0, n).toCharArray();
        for (int i = n; i <= data.length(); i++) {
            if (rem[0] == '1') {
                for (int j = 0; j < n; j++)
                    rem[j] = xor(rem[j], key.charAt(j));
            }
        }
    }
}
```

```

// shift left by one bit, bring next data bit if (i < data.length()) {
for (int j = 0; j < n - 1; j++) rem[j] = rem[j + 1];
rem[n - 1] = data.charAt(i);
} else {
for (int j = 0; j < n - 1; j++) rem[j] = rem[j + 1];
}
}
return new String(rem, 0, n - 1);
}

static char xor(char a, char b) { return (a == b) ? '0' : '1';
}

public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);

System.out.print("Enter Data bits (e.g. 1101011011): ");
String data = sc.next();
System.out.print("Enter Generator/Key (e.g. 10011): ");
String key = sc.next();

int n = key.length();
String appendedData = data + "0".repeat(n - 1);

// Step 1: Sender computes CRC
String crc = xorDivision(appendedData, key);
String codeword = data + crc;
System.out.println("\n---- SENDER SIDE      ");
System.out.println("CRC (Checksum) generated : " + crc);
System.out.println("Codeword transmitted      : " + codeword);

// Step 2: Simulate transmission (user can introduce an error)
System.out.print("\nEnter received codeword (press Enter to keep it "
+ "unchanged,\nor type a corrupted version to test error "
+ "detection): "); sc.nextLine();
String received = sc.nextLine().trim();
if (received.isEmpty()) received = codeword;

// Step 3: Receiver divides received codeword by same key
String remainder = xorDivision(received, key);
System.out.println("\n---- RECEIVER SIDE    ");

System.out.println("Received codeword : " + received);
System.out.println("Remainder obtained : " + remainder);

if (remainder.matches("0+")) {

```

```
System.out.println("Result: No error detected. Data accepted.");
} else {
System.out.println("Result: Error detected in received data!");
}
sc.close();
}
}
```

SAMPLE OUTPUT

```
Enter Data bits (e.g. 1101011011): 1101011011
Enter Generator/Key (e.g. 10011): 10011

---- SENDER SIDE
CRC (Checksum) generated : 1110
Codeword transmitted      : 11010110111110

Enter received codeword (press Enter to keep it unchanged,
or type a corrupted version to test error detection):

---- RECEIVER SIDE
Received codeword : 11010110111110
Remainder obtained : 0000
Result: No error detected. Data accepted.
```

PRE LAB QUESTIONS

1. State the purpose of CRC in data communication.
2. Define a generator polynomial and give one example (e.g. CRC-CCITT, CRC-32).
3. Justify why XOR is used instead of normal binary subtraction in CRC division.
4. Determine the relationship between the number of bits in the key and the number of CRC bits appended to the data.
5. How the minimum Hamming distance relates to error detection capability.

POST LAB QUESTIONS

1. Verify manually why the codeword 11010110111110 gives a zero remainder.
2. Assess whether CRC can detect all possible burst errors, and justify your answer.
3. Modify the program to handle the case where the generator key itself is chosen incorrectly (e.g., all zeros).
4. Compare CRC with the Checksum method of error detection in terms of accuracy and computation.
5. Differentiate between error detection and error correction, with one example of each.

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT :

Thus a Java program was developed to detect errors in transmitted data using the Cyclic Redundancy Check technique. The program was executed successfully, correctly accepting an error-free codeword and correctly flagging a corrupted codeword as erroneous.

AIM

To write a Java program to implement the flow control mechanism using the Sliding Window Protocol.

ALGORITHM

1. Start the program.
2. Read the total number of frames to send and the sliding window size.
3. Initialize sent = 0 and acked = 0.
4. Repeat the following steps while acked is less than total frames.
5. Determine the current window as the range [acked, acked + windowSize).
6. Sender transmits every frame in the current window and increments sent.
7. Receiver acknowledges each transmitted frame in order and increments acked.
8. Slide the window forward once all frames in it are acknowledged.
9. Display each frame transmission and acknowledgement as it happens.
10. Stop the program when all frames are delivered and acknowledged.

PROGRAM

```
import java.util.Scanner;

/*
 * Experiment 3: Design and Develop a Program for Flow Control
 * Mechanisms Using Sliding Window Protocol.
 * The sender can have multiple frames "in flight" at once, up to the window size, without waiting for
 * each individual acknowledgement.
 * This improves throughput compared to Stop-and-Wait.
 */
public class SlidingWindow {

    public static void main(String[] args) throws InterruptedException {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter total number of frames to send: ");
        int totalFrames = sc.nextInt();

        System.out.print("Enter sliding window size: ");
        int windowSize = sc.nextInt();

        int sent = 0; // frames sent so far
        int acked = 0; // frames acknowledged so far
```

```

System.out.println("\n---- SLIDING WINDOW SIMULATION      ");

while (acked < totalFrames)
{
    System.out.println("\nCurrent Window: Frames " + acked + " to " + (Math.min(acked + windowSize,
totalFrames) - 1));

    // Sender transmits all frames within the current window
    int windowEnd = Math.min(acked + windowSize, totalFrames);
    for (int i = sent; i < windowEnd; i++)
    {
        System.out.println("Sender -> Sending Frame " + i); sent++;
    }

    // Receiver acknowledges frames one by one (cumulative ACK)
    for (int i = acked; i < windowEnd; i++) {
        System.out.println("Receiver -> ACK received for Frame " + i);
        acked++;
    }

    System.out.println("Window slides forward. Frames acked so far = "+ acked);
}

System.out.println("\nAll " + totalFrames + " frames delivered "
+ "successfully using a window size of "
+ windowSize + ".");
sc.close();
}
}

```

SAMPLE OUTPUT

```

Enter total number of frames to send: 8
Enter sliding window size: 4

---- SLIDING WINDOW SIMULATION

Current Window: Frames 0 to 3
Sender   -> Sending Frame 0
Receiver -> ACK received for Frame 0
Window slides forward. Frames acked so far = 1
Sender   -> Sending Frame 1
Receiver -> ACK received for Frame 1
Window slides forward. Frames acked so far = 2
Sender   -> Sending Frame 2
Receiver -> ACK received for Frame 2
Window slides forward. Frames acked so far = 3

```

```
...
Sender    -> Sending Frame 7
Receiver  -> ACK received for Frame 7
Window slides forward. Frames acked so far = 8

All 8 frames delivered successfully.
```

PRE LAB QUESTIONS

1. Why flow control is needed between a fast sender and a slow receiver.
2. Define 'window size' in the Sliding Window Protocol.
3. Differentiate between the Go-Back-N and Selective Repeat sliding window variants.
4. Predict the effect on throughput when the window size is increased.
5. Describe the term 'cumulative acknowledgement' as used in the Sliding Window Protocol.

POST LAB QUESTIONS

1. Account for frames 8 and 9 being sent together as a window of size 2 instead of 4, based on the sample output.
2. Identify which protocol the Sliding Window reduces to when the window size equals 1, and justify your answer.
3. Modify this program to simulate a lost acknowledgement inside a window.
4. Relate window size to the bandwidth-delay product of a link.
5. Calculate the number of windows required to send 10 frames if the window size is changed to 4

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT :

Thus a Java program was developed to implement flow control using the Sliding Window Protocol. The program was executed successfully and all 10 frames were transmitted and acknowledged correctly using a window size of 4.

EX.NO.: 4	STOP-AND-WAIT ARQ PROTOCOL FOR RELIABLE DATA TRANSMISSION
DATE:	

AIM

To write a Java program to implement the Stop-and-Wait ARQ (Automatic Repeat reQuest) protocol for reliable data transmission.

ALGORITHM

1. Start the program.
2. Read the total number of frames to send and the probability of frame loss (%).
3. For each frame from 0 to totalFrames - 1, repeat steps 4 to 8.
4. Sender transmits the current frame and starts a timer.
5. Randomly decide, based on the loss probability, whether the frame is lost in transit.
6. If the frame is lost, a timeout occurs and the sender retransmits the same frame.
7. If the frame is received correctly, the receiver sends an ACK for that frame.
8. On receiving the ACK, the sender proceeds to the next frame.
9. Display each transmission attempt, loss/timeout event, and acknowledgement.
10. Stop the program once all frames are successfully acknowledged.

PROGRAM

```
import java.util.Random; import java.util.Scanner;

/*
 * Experiment 4: Create a Program to Implement Stop-and-Wait ARQ
 * Protocol for Reliable Data Transmission.
 * The sender transmits ONE frame and then waits for its ACK before sending the next frame. If the ACK
 * does not arrive (frame or ACK lost) within the timeout, the sender retransmits the same frame.
 */
public class StopAndWaitARQ {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random random = new Random();

        System.out.print("Enter total number of frames to send: ");
        int totalFrames = sc.nextInt();

        System.out.print("Enter probability of frame loss in % (0-100): ");
        int lossPercent = sc.nextInt();

        System.out.println("\n---- STOP AND WAIT ARQ SIMULATION    ");

        for (int frame = 0; frame < totalFrames; frame++)
        {
```

```

boolean ackReceived = false;
int attempt = 0;

while (!ackReceived) { attempt++;
System.out.println("\nSender -> Sending Frame " + frame + " (Attempt " + attempt + ")");
int chance = random.nextInt(100);
if (chance < lossPercent)
{
System.out.println("Network -> Frame " + frame
+ " LOST in transit!");
System.out.println("Sender -> Timeout occurred. "
+ "Retransmitting...");
} else
{
System.out.println("Receiver -> Frame " + frame+ " received correctly.");
System.out.println("Receiver -> Sending ACK" + frame);
System.out.println("Sender -> ACK" + frame+ " received. Moving to next frame.");
ackReceived = true;
}
}
}
System.out.println("\nAll " + totalFrames + " frames delivered "+ "reliably using Stop-and-Wait ARQ.");
sc.close();
}
}

```

SAMPLE OUTPUT

```

Enter total number of frames to send: 5
Enter probability of frame loss in % (0-100): 20

---- STOP AND WAIT ARQ SIMULATION

Sender -> Sending Frame 0
Receiver -> Frame 0 received correctly.
Receiver -> Sending ACK0
Sender -> ACK0 received. Moving to next frame.

Sender -> Sending Frame 1
Network -> Frame 1 LOST in transit!
Sender -> Timeout occurred. Retransmitting Frame 1
Sender -> Sending Frame 1
Receiver -> Frame 1 received correctly.
Receiver -> Sending ACK1
Sender -> ACK1 received. Moving to next frame.

Sender -> Sending Frame 2
Receiver -> Frame 2 received correctly.

```

```
Receiver -> Sending ACK2
Sender   -> ACK2 received. Moving to next frame.

All 5 frames delivered successfully.
```

PRE LAB QUESTIONS

1. Expand the term ARQ and state its purpose.
2. Justify why the sender needs to start a timer after sending a frame in Stop-and-Wait ARQ.
3. Identify the main drawback of Stop-and-Wait ARQ in terms of channel utilization.
4. Differentiate between a lost frame and a lost acknowledgement, and state how each affects the sender.
5. Describe 'sequence numbering' and explain why Stop-and-Wait ARQ needs only 1 bit for it.

POST LAB QUESTIONS

1. Calculate the total number of transmission attempts made for all 5 frames, based on the sample output.
2. Predict the program's behaviour if the loss probability was set to 100%, and modify the code to avoid an infinite loop in that case.
3. Compare the efficiency of Stop-and-Wait ARQ with the Sliding Window Protocol for a link with high bandwidth-delay product.
4. Illustrate how a 'duplicate ACK' can occur if the sender's timeout value is too small.
5. Compute the efficiency of Stop-and-Wait ARQ if the transmission time is 1 ms and the round-trip propagation delay is 20 ms.

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT:

Thus a Java program was developed to implement the Stop-and-Wait ARQ protocol for reliable data transmission. The program was executed successfully, and reliable delivery of all frames was verified even with simulated frame loss and retransmission.

AIM

To write a Java program to implement Distance Vector Routing (used by the RIP protocol) and to compute the shortest distance from every router to every other router.

ALGORITHM

1. Start the program.
2. Read the number of routers (n) and the direct-link cost matrix (999 = no direct link).
3. Initialize the distance table $\text{dist}[i][j]$ with the direct cost matrix values.
4. Repeat steps 5 and 6 until no further improvement is possible.
5. For every pair of routers (i, j) and intermediate router k, check if $\text{dist}[i][k] + \text{dist}[k][j]$ is less than $\text{dist}[i][j]$.
6. If so, update $\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$ (a shorter path was learned via neighbour k).
7. Display the distance table after every round of updates.
8. Continue until the distance tables converge (no updates happen in a full round).
9. Display the final converged distance vector table for all routers.
10. Stop the program.

PROGRAM

```
import java.util.Scanner;
/*
 * Experiment 5: Write a Program to Implement Distance Vector
 * Routing Using the RIP Protocol.
 * Distance Vector Routing (used by RIP) works on the Bellman-Ford algorithm: every router keeps a
 * table of the shortest known distance to every other router and shares this table with its neighbours. Each
 * router updates its table if a neighbour offers a shorter path:
 *  $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$ 
 */
public class DistanceVectorRouting {
    static final int INF = 999;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of routers: ");
        int n = sc.nextInt();
        int[][] cost = new int[n][n];
        System.out.println("Enter the cost matrix (use " + INF + " for no direct link, 0 for diagonal):");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                cost[i][j] = sc.nextInt();
        // Distance vector table, initialised with direct link costs
        int[][] dist = new int[n][n];
        for (int i = 0; i < n; i++) dist[i] = cost[i].clone();
    }
}
```

```

System.out.println("\nInitial Distance Vector Tables:");
printTables(dist, n);
// Bellman-Ford style updates: each router learns from every neighbour boolean updated = true;
int round = 1; while (updated)
{
    updated = false;
    for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) {
    for (int k = 0; k < n; k++) {
    if (dist[i][k] + dist[k][j] < dist[i][j]) {
    dist[i][j] = dist[i][k] + dist[k][j]; updated = true;
    }
    }
    }
    }
    if (updated) {
    System.out.println("\nAfter Update Round " + round + ":");
    printTables(dist, n);
    round++;
    }
    }
    System.out.println("\nFinal Converged Distance Vector "+ "(Shortest Cost) Tables:");
    printTables(dist, n);
    sc.close();
}

static void printTables(int[][] dist, int n) {
    System.out.print(" ");
    for (int j = 0; j < n; j++) System.out.print("R" + j + " ");
    System.out.println();
    for (int i = 0; i < n; i++) { System.out.print("R" + i + " : ");
    for (int j = 0; j < n; j++)
    System.out.printf("%-4d", dist[i][j]);
    System.out.println();
    }
}
}
}

```

SAMPLE OUTPUT

```

Enter number of routers: 4
Enter the cost matrix (use 999 for no direct link):
R0: 0 2 999 7
R1: 2 0 1 999
R2: 999 1 0 3
R3: 7 999 3 0

Initial Distance Vector Tables:
    R0    R1    R2    R3

```

```

R0 : 0    2    999  7
R1 : 2    0    1    999
R2 : 999  1    0    3
R3 : 7    999  3    0

```

After Update Round 1:

```

      R0    R1    R2    R3
R0 : 0    2    3    7
R1 : 2    0    1    4
R2 : 3    1    0    3
R3 : 7    4    3    0

```

After Update Round 2:

```

      R0    R1    R2    R3
R0 : 0    2    3    6
R1 : 2    0    1    4
R2 : 3    1    0    3
R3 : 6    4    3    0

```

Final Converged Distance Vector (Shortest Cost) Tables:

```

      R0    R1    R2    R3
R0 : 0    2    3    6
R1 : 2    0    1    4
R2 : 3    1    0    3
R3 : 6    4    3    0

```

PRE LAB QUESTIONS

1. Identify the algorithm that forms the basis of Distance Vector Routing.
2. State the information a router shares with its neighbours in Distance Vector Routing.
3. Explain the 'count-to-infinity' problem in RIP and why it occurs.
4. State the maximum hop count allowed by RIP and justify why this limit is imposed.
5. Identify the routing metric that RIP uses to select the best path.

POST LAB QUESTIONS

1. Trace how Router R0 arrived at a shortest cost of 2 to R2 instead of the direct cost 999, based on the sample output.
2. Determine the path Router R0 uses to reach Router R2, based on the final table.
3. Account for Distance Vector Routing usually converging slower than Link State Routing.
4. Suggest one technique (e.g., split horizon, poison reverse) used by RIP to reduce routing loops.
5. Predict whether any entry in the final table would change if the link cost between R1 and R2 increases from 3 to 10, and justify your answer.

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT :

Thus a Java program was developed to implement Distance Vector Routing based on the RIP protocol. The program was executed successfully, and the converged shortest-distance table for all routers was obtained after the required update rounds.

AIM

To write a Java program to implement Link State Routing (used by the OSPF protocol) using Dijkstra's shortest path algorithm.

ALGORITHM

1. Start the program.
2. Read the number of routers (n), the link cost matrix, and the source router.
3. Initialize dist[] with direct costs from the source; mark only the source as visited.
4. Repeat steps 5 and 6 a total of (n - 1) times.
5. Select the unvisited router u with the minimum current distance and mark it visited.
6. For every unvisited neighbour v of u, if $\text{dist}[u] + \text{cost}[u][v]$ is less than $\text{dist}[v]$, update $\text{dist}[v]$ and record u as the previous hop of v.
7. After all routers are visited, dist[] contains the shortest cost from the source to every router.
8. Reconstruct the shortest path to each destination using the recorded previous-hop values.
9. Display the shortest path tree (destination, cost, and path).
10. Stop the program.

PROGRAM

```
import java.util.Scanner;

/*
 * Experiment 6: Construct a Program to Implement Link State
 * Routing Using the OSPF Protocol.
 * OSPF uses Link State Routing, which is based on Dijkstra's shortest Path Algorithm.
 * Every router floods its link costs to all other routers so that every router builds the SAME complete
 * map (link state database) of the network, and independently computes the shortest path tree from itself
 * to every other router.
 */
public class LinkStateRouting {
    static final int INF = 999;
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of routers: ");
        int n = sc.nextInt();
        int[][] cost = new int[n][n];
        System.out.println("Enter the link cost matrix (use " + INF + " for no direct link, 0 for diagonal):");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++) cost[i][j] = sc.nextInt();
        System.out.print("Enter the source router number (0 to " + (n - 1) + "): ");
```

```

    int src = sc.nextInt();
    dijkstra(cost, src, n); sc.close();
}
static void dijkstra(int[][] cost, int src, int n)
{
    int[] dist = new int[n];
    boolean[] visited = new boolean[n]; int[] prev = new int[n];

    for (int i = 0; i < n; i++) { dist[i] = cost[src][i];
    prev[i] = (dist[i] != INF && i != src) ? src : -1;
    }
    visited[src] = true; dist[src] = 0;

    for (int count = 1; count < n; count++) { int u = -1, min = INF + 1;
    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] < min) { min = dist[i];
        u = i;
        }
    }
    if (u == -1) break; // remaining routers unreachable visited[u] = true;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && cost[u][v] != INF && dist[u] + cost[u][v] < dist[v]) {
            dist[v] = dist[u] + cost[u][v]; prev[v] = u;
        }
    }
    }
    System.out.println("\nShortest Path Tree computed by Router "+ src + " (Link State / OSPF:");
    System.out.println("Destination\tCost\tPath"); for (int i = 0; i < n; i++) {
    if (i == src)
        continue;
    System.out.print("R" + i + "\t\t" + dist[i] + "\t" + buildPath(prev, src, i));
    System.out.println();
    }
    static String buildPath(int[] prev, int src, int dest) {
        StringBuilder path = new StringBuilder("R" + dest);
        int cur = dest;
        while (cur != src && prev[cur] != -1) { cur = prev[cur];
        path.insert(0, "R" + cur + " -> ");
        }
        return path.toString();
    }
}

```

SAMPLE OUTPUT

```
Enter number of routers: 4
Enter the link cost matrix (use 999 for no direct link):
R0: 0 2 999 7
R1: 2 0 1 999
R2: 999 1 0 3
R3: 7 999 3 0
Enter the source router number (0 to 3): 0

Shortest Path Tree computed by Router 0 (Link State / OSPF):
Destination  Cost  Path
R0           0      0
R1           2      0 -> 1
R2           3      0 -> 1 -> 2
R3           6      0 -> 1 -> 2 -> 3
```

PRE LAB QUESTIONS

1. Identify the algorithm OSPF uses internally to compute the shortest path tree.
2. Define a Link State Advertisement (LSA) and state what it contains.
3. Differentiate between how Link State and Distance Vector routers build their routing information.
4. Justify why OSPF is called a 'link state' protocol while RIP is called a 'distance vector' protocol.
5. Describe the term 'flooding' as used in Link State Routing.

POST LAB QUESTIONS

1. Verify that the path R0 -> R3 -> R2 (cost 2) is indeed shorter than any direct path from R0 to R2, based on the sample output.
2. Justify why every router in Link State Routing needs a complete copy of the network topology (link state database).
3. Predict how the shortest path tree would change if the cost of the link between R0 and R3 increased from 1 to 5.
4. Compare the convergence speed and scalability of OSPF (Link State) with RIP (Distance Vector) in large networks.
5. Determine the time complexity of Dijkstra's algorithm used in this program, in terms of the number of routers n.

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT :

Thus a Java program was developed to implement Link State Routing based on the OSPF protocol using Dijkstra's shortest path algorithm. The program was executed successfully, and the shortest path tree from the source router to all destination routers was obtained correctly.

EX.NO.:7	CLIENT-SERVER COMMUNICATION USING TCP FOR DATE AND TIME SERVICE
DATE:	

AIM

To develop a Java program to implement client-server communication using TCP sockets, where the server sends the current date and time to a connecting client.

ALGORITHM

1. Import required packages: `java.net.\*`, `java.io.\*`, `java.util.Date`.
2. Create a `ServerSocket` object bound to a fixed port number (e.g., 5000).
3. Print a message indicating the server is waiting for a client connection.
4. Call `accept()` on the server socket to block until a client connects; store the returned `Socket`.
5. Obtain the output stream of the accepted socket and wrap it in a `PrintWriter`.
6. Create a `Date` object to capture the current system date and time.
7. Convert the `Date` object to a string and send it to the client through the `PrintWriter`.
8. Create a `Socket` object connecting to the server's IP address/hostname and port number.
9. Obtain the input stream of the socket and wrap it in a `BufferedReader`.
10. Display the received date and time on the console and Close the socket and terminate the program.

PROGRAM

TCPDateServer.java

```
import java.io.*;
import java.net.*;
import java.util.Date;

public class TCPDateServer {
    public static void main(String[] args) {
        int port = 5000;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Date-Time Server started. Waiting for client on port " + port + " ...");
            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("Client connected: " + clientSocket.getInetAddress());
```

```

PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
Date currentDate = new Date();
out.println(currentDate.toString());

clientSocket.close();
System.out.println("Response sent. Connection closed.\n");
}
} catch (IOException e) {
System.out.println("Server error: " + e.getMessage());
}
}
}

```

TCPDateClient.java

```

import java.io.*;
import java.net.*;

public class TCPDateClient {
    public static void main(String[] args) {
        String serverAddress = "127.0.0.1";
        int port = 5000;
        try (Socket socket = new Socket(serverAddress, port)) {
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            String serverTime = in.readLine();
            System.out.println("Date and Time received from server: " + serverTime);
        } catch (IOException e) {
            System.out.println("Client error: " + e.getMessage());
        }
    }
}

```

SAMPLE OUTPUT

```

Server side:
Date-Time Server started. Waiting for client on port 5000 ...
Client connected: /127.0.0.1
Response sent. Connection closed.
Client side:
Date and Time received from server: Sun Jul 05 11:42:18 IST 2026

```

PRE LAB QUESTIONS

1. What is the difference between TCP and UDP, and why is TCP suitable for a date-time service?
2. What is a socket, and what role does the `ServerSocket` class play in Java networking?
3. Explain the three-way handshake used to establish a TCP connection.
4. What is the purpose of the `accept()` method in server-side socket programming?
5. Why must the client know the server's IP address and port number in advance?

POST LAB QUESTIONS

1. What would happen if the server tries to bind to a port that is already in use?
2. How can the server be modified to handle multiple clients concurrently?
3. What is the significance of closing sockets after communication is complete?
4. How does `PrintWriter`'s autoflush parameter affect data transmission?
5. How would you modify the program to send the time in a custom formatted string instead of the default `Date.toString()` output?

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT

The Java program to implement client-server communication using TCP sockets for a date and time service was developed and executed successfully. The server correctly transmitted the current system date and time to the connecting client over a TCP connection.

EX.NO.:8	SIMPLE TFTP CLIENT-SERVER APPLICATION USING RRQ AND WRQ OPERATIONS
DATE:	

AIM

To build a Java program that implements a simplified TFTP (Trivial File Transfer Protocol) client-server application demonstrating Read Request (RRQ) and Write Request (WRQ) operations using UDP sockets.

ALGORITHM

Server:

1. Import ``java.net.*`` and ``java.io.*`` packages.
2. Create a ``DatagramSocket`` bound to a well-known TFTP-style port (e.g., 6900).
3. Wait continuously in a loop to receive incoming ``DatagramPacket``s from clients.
4. On receiving a packet, extract the opcode (first bytes) to determine whether it is an RRQ or a WRQ.
5. If the opcode indicates RRQ, extract the requested filename, read the file's content (or simulated content) from the server's storage.
6. Send the file data back to the client's address and port using a new ``DatagramPacket``.
7. If the opcode indicates WRQ, extract the filename and prepare to receive data blocks from the client.
8. Receive the subsequent data packet(s) from the client and write/store the received content on the server side.
9. Send an acknowledgment (ACK) packet back to the client after processing each request.
10. Repeat the loop to serve further requests, or close the socket to terminate the server.

Client:

1. Create a ``DatagramSocket`` for communication.
2. Construct a request packet with an opcode (1 = RRQ, 2 = WRQ) followed by the filename.
3. Send the request packet to the server's IP address and port.
4. If performing an RRQ, wait to receive the data packet from the server and display/save the received content.
5. If performing a WRQ, send the file data packet immediately after the request and wait for the server's ACK.
6. Close the socket and terminate the program.

PROGRAM

TFTPServer.java

```
import java.net.*;
import java.util.*;

public class TFTPServer {
    static final int RRQ = 1;
    static final int WRQ = 2;
    static Map<String, String>fileStore = new HashMap<>();

    public static void main(String[] args) {
        int port = 6900;
        fileStore.put("sample.txt", "Hello, this is sample TFTP file content.");

        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("TFTP Server started on port " + port);
            byte[] buffer = new byte[1024];

            while (true) {
                DatagramPacketrequestPacket = new DatagramPacket(buffer, buffer.length);
                socket.receive(requestPacket);

                InetAddressclientAddress = requestPacket.getAddress();
                int clientPort = requestPacket.getPort();
                String data = new String(requestPacket.getData(), 0, requestPacket.getLength());

                int opcode = Integer.parseInt(data.substring(0, 1));
                String filename = data.substring(1);

                if (opcode == RRQ) {
                    System.out.println("RRQ received for file: " + filename);
                    String fileContent = fileStore.getOrDefault(filename, "ERROR: File not found");
                    byte[] responseData = fileContent.getBytes();
                    DatagramPacketresponsePacket = new DatagramPacket(responseData, responseData.length,
                        clientAddress, clientPort);
```

```

socket.send(responsePacket);
} else if (opcode == WRQ) {
System.out.println("WRQ received for file: " + filename);
byte[] ackBuffer = new byte[1024];
DatagramPacketackRequest = new DatagramPacket(ackBuffer, ackBuffer.length);
socket.receive(ackRequest);
String fileData = new String(ackRequest.getData(), 0, ackRequest.getLength());
fileStore.put(filename, fileData);
System.out.println("File stored: " + filename);

byte[] ackMsg = "ACK".getBytes();
DatagramPacketackPacket = new DatagramPacket(ackMsg, ackMsg.length, clientAddress, clientPort);
socket.send(ackPacket);
}
}
} catch (Exception e) {
System.out.println("Server error: " + e.getMessage());
}
}
}

```

TFTPClient.java

```

import java.net.*;
import java.util.*;

public class TFTPClient {
static final int RRQ = 1;
static final int WRQ = 2;

public static void main(String[] args) throws Exception {
DatagramSocket socket = new DatagramSocket();
InetAddressserverAddress = InetAddress.getByName("127.0.0.1");
int serverPort = 6900;

Scanner sc = new Scanner(System.in);
System.out.println("Enter operation (1-RRQ, 2-WRQ): ");
int choice = sc.nextInt();

```

```

sc.nextLine();
System.out.println("Enter filename: ");
String filename = sc.nextLine();

if (choice == RRQ) {
String request = RRQ + filename;
byte[] requestData = request.getBytes();
DatagramPacketrequestPacket = new DatagramPacket(requestData, requestData.length, serverAddress,
    serverPort);
socket.send(requestPacket);

byte[] buffer = new byte[1024];
DatagramPacketresponsePacket = new DatagramPacket(buffer, buffer.length);
socket.receive(responsePacket);
String fileContent = new String(responsePacket.getData(), 0, responsePacket.getLength());
System.out.println("File content received:\n" + fileContent);

} else if (choice == WRQ) {
System.out.println("Enter file content to upload: ");
String content = sc.nextLine();

String request = WRQ + filename;
byte[] requestData = request.getBytes();
DatagramPacketrequestPacket = new DatagramPacket(requestData, requestData.length, serverAddress,
    serverPort);
socket.send(requestPacket);

byte[] contentData = content.getBytes();
DatagramPacketdataPacket = new DatagramPacket(contentData, contentData.length, serverAddress,
    serverPort);
socket.send(dataPacket);

byte[] ackBuffer = new byte[1024];
DatagramPacketackPacket = new DatagramPacket(ackBuffer, ackBuffer.length);
socket.receive(ackPacket);
String ackMsg = new String(ackPacket.getData(), 0, ackPacket.getLength());
System.out.println("Server response: " + ackMsg);

```

```
}  
socket.close();  
}  
}
```

SAMPLE OUTPUT

```
Server side:  
TFTP Server started on port 6900  
RRQ received for file: sample.txt  
  
Client side (RRQ):  
Enter operation (1-RRQ, 2-WRQ):  
1  
Enter filename:  
sample.txt  
File content received:  
Hello, this is sample TFTP file content.  
  
Client side (WRQ):  
Enter operation (1-RRQ, 2-WRQ):  
2  
Enter filename:  
newfile.txt  
Enter file content to upload:  
Networking Lab WRQ test
```

PRE LAB QUESTIONS

1. What is TFTP, and how does it differ from FTP in terms of complexity and transport protocol used?
2. Why does TFTP use UDP instead of TCP as its transport-layer protocol?
3. What are the standard opcodes used in TFTP messages (RRQ, WRQ, DATA, ACK, ERROR)?
4. What is the typical default port number used by the TFTP protocol?
5. What are the limitations of TFTP compared to standard FTP (e.g., authentication, directory listing)?

POST LAB QUESTIONS

1. How would you extend this program to transfer files larger than a single UDP packet (block-based transfer)?
2. What issues can arise from using UDP for file transfer, and how does real TFTP handle packet loss?
3. How would you implement proper error handling using TFTP's ERROR opcode?
4. What changes are needed to support transferring actual files from disk instead of in-memory strings?
5. Why is TFTP commonly used for booting diskless workstations and network device firmware updates?

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT

A simplified Java program simulating TFTP client-server communication using UDP sockets was implemented and tested successfully, demonstrating both Read Request (RRQ) and Write Request (WRQ) operations

EX.NO.:9	CLIENT-SERVER COMMUNICATION USING UDP SOCKETS
DATE:	

AIM

To construct a Java program to implement client-server communication using UDP (User Datagram Protocol) sockets.

ALGORITHM

Server:

1. Import `java.net.\*` package for socket classes.
2. Create a `DatagramSocket` bound to a specific port (e.g., 7000).
3. Create a byte array buffer to hold incoming data.
4. Wrap the buffer in a `DatagramPacket` and call `receive()` to wait for a client message.
5. Extract and display the message content and the sender's address/port from the received packet.
6. Prepare a reply message as a byte array.
7. Create a new `DatagramPacket` with the reply, addressed to the client's IP and port.
8. Send the reply packet back to the client using `send()`.
9. Loop back to step 4 to continue serving further messages, or exit if a termination message (e.g., "exit") is received.
10. Close the socket and terminate the program.

Client:

1. Create a `DatagramSocket` for the client (no need to bind to a specific port).
2. Read a message to send from the user via console input.
3. Convert the message to a byte array and wrap it in a `DatagramPacket` addressed to the server's IP and port.
4. Send the packet using `send()`.
5. Receive the server's reply using a new `DatagramPacket` and `receive()`.
6. Display the reply message on the console; close the socket when done.

PROGRAM

UDPServer.java

```
import java.net.*;

public class UDPServer {
    public static void main(String[] args) {
```

```

int port = 7000;
try (DatagramSocket socket = new DatagramSocket(port)) {
    System.out.println("UDP Server started on port " + port);
    byte[] buffer = new byte[1024];

    while (true) {
        DatagramPacket requestPacket = new DatagramPacket(buffer, buffer.length);
        socket.receive(requestPacket);

        String message = new String(requestPacket.getData(), 0, requestPacket.getLength());
        InetAddress clientAddress = requestPacket.getAddress();
        int clientPort = requestPacket.getPort();
        System.out.println("Received from client: " + message);

        if (message.equalsIgnoreCase("exit")) {
            System.out.println("Termination message received. Server shutting down.");
            break;
        }

        String replyMessage = "Server received: " + message;
        byte[] replyData = replyMessage.getBytes();
        DatagramPacket replyPacket = new DatagramPacket(replyData, replyData.length, clientAddress, clientPort);
        socket.send(replyPacket);
    }
} catch (Exception e) {
    System.out.println("Server error: " + e.getMessage());
}
}

```

UDPClient.java

```

import java.net.*;
import java.util.Scanner;
public class UDPClient {
    public static void main(String[] args) throws Exception {
        DatagramSocket socket = new DatagramSocket();
        InetAddress serverAddress = InetAddress.getByName("127.0.0.1");

```

```

int serverPort = 7000;
Scanner sc = new Scanner(System.in);
System.out.println("Enter message to send to server (type 'exit' to quit): ");
String message = sc.nextLine();
byte[] messageData = message.getBytes();
DatagramPacket requestPacket = new DatagramPacket(messageData, messageData.length,
serverAddress, serverPort);
socket.send(requestPacket);
if (!message.equalsIgnoreCase("exit")) {
byte[] buffer = new byte[1024];
DatagramPacket replyPacket = new DatagramPacket(buffer, buffer.length);
socket.receive(replyPacket);
String reply = new String(replyPacket.getData(), 0, replyPacket.getLength());
System.out.println("Server reply: " + reply);
}
socket.close();
}
}

```

SAMPLE OUTPUT

```

Server side:
TFTP Server started on port 6900
RRQ received for file: sample.txt

Client side (RRQ):
Enter operation (1-RRQ, 2-WRQ):
1
Enter filename:
sample.txt
File content received:
Hello, this is sample TFTP file content.

Client side (WRQ):
Enter operation (1-RRQ, 2-WRQ):
2
Enter filename:
newfile.txt
Enter file content to upload:
Networking Lab WRQ test

```

PRE LAB QUESTIONS

1. What is UDP, and how does it differ from TCP in terms of connection and reliability?
2. Why is UDP called a "connectionless" protocol?

3. What is the role of a `DatagramPacket` in Java's UDP socket programming?
4. In what real-world applications is UDP preferred over TCP, and why?
5. What are the risks of packet loss and out-of-order delivery when using UDP?

POST LAB QUESTIONS

1. How would you modify this program to allow continuous two-way conversation instead of a single request-reply?
2. What happens if the server is not running when the client attempts to send a message?
3. How can you make the UDP server handle multiple clients simultaneously?
4. What is the maximum practical size of a UDP datagram, and how would you handle larger messages?
5. Compare the reliability guarantees of this program with what a TCP-based equivalent would provide.

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT

The Java program implementing client-server communication using UDP sockets was developed and executed successfully, demonstrating connectionless message exchange between client and server

EX.NO.:10	DOMAIN NAME RESOLUTION USING THE DOMAIN NAME SYSTEM (DNS)
DATE:	

AIM

To create a Java program to implement and analyze domain name resolution using the Domain Name System (DNS), performing both forward (hostname to IP) and reverse (IP to hostname) lookups.

ALGORITHM

1. Import the `java.net.InetAddress` and `java.net.UnknownHostException` classes.
2. Prompt the user to enter a domain name (e.g., "www.google.com") or an IP address.
3. Call `InetAddress.getByName(domainName)` to resolve the hostname to its corresponding IP address.
4. Use `getAllByName(domainName)` to retrieve all IP addresses associated with the domain, in case of multiple records.
5. Iterate through the returned array of `InetAddress` objects and print each resolved IP address.
6. Measure and record the time taken for the DNS resolution using `System.currentTimeMillis()` before and after the call.
7. For reverse lookup, take the resolved IP address and call `getCanonicalHostName()` on the `InetAddress` object.
8. Display the canonical hostname obtained from the reverse lookup.
9. Handle `UnknownHostException` gracefully if the domain name or IP address cannot be resolved.
10. Display all analysis results (IP addresses, hostname, resolution time) and terminate the program.

PROGRAM

DNSResolver.java

```
import java.net.*;
import java.util.Scanner;

public class DNSResolver {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter domain name (e.g., www.google.com): ");
        String domainName = sc.nextLine();

        try {
            long startTime = System.currentTimeMillis();

            InetAddress address = InetAddress.getByName(domainName);
```

```

InetAddress[] allAddresses = InetAddress.getAllByName(domainName);

long endTime = System.currentTimeMillis();

System.out.println("\n--- DNS Resolution Result ---");
System.out.println("Domain Name: " + domainName);
System.out.println("Primary IP Address: " + address.getHostAddress());

System.out.println("\nAll associated IP addresses:");
for (InetAddress addr : allAddresses) {
    System.out.println(" " + addr.getHostAddress());
}

System.out.println("\nCanonical Host Name (Reverse Lookup): " + address.getCanonicalHostName());
System.out.println("Time taken for resolution: " + (endTime - startTime) + " ms");

} catch (UnknownHostException e) {
    System.out.println("Error: Unable to resolve domain name - " + domainName);
}
}
}

```

SAMPLE OUTPUT

```

Enter domain name (e.g., www.google.com):
www.google.com

--- DNS Resolution Result ---
Domain Name: www.google.com
Primary IP Address: 142.250.196.100

All associated IP addresses:
142.250.196.100
142.250.196.101
142.250.196.102

Canonical Host Name (Reverse Lookup): maa03s10-in-f4.1e100.net
Time taken for resolution: 42 ms

```

PRE LAB QUESTIONS

1. What is the purpose of the Domain Name System (DNS) in computer networks?
2. What is the difference between a forward lookup and a reverse lookup in DNS?

3. What is a canonical hostname, and how does it differ from an alias?
4. What DNS record types (A, AAAA, CNAME, MX, NS) are commonly used, and what does each represent?
5. What is the role of a DNS resolver and DNS caching in reducing lookup time?

POST LAB QUESTIONS

1. Why might a single domain name resolve to multiple IP addresses?
2. How does DNS caching affect the resolution time observed in repeated queries?
3. What could cause an `UnknownHostException` to be thrown during resolution?
4. How would you modify the program to query a specific DNS server rather than the system default?
5. Explain how DNS resolution contributes to load balancing for large-scale web services.

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT

The Java program to implement and analyze domain name resolution using DNS was developed and executed successfully, demonstrating forward and reverse lookups along with resolution timing analysis

EX.NO.:11	CLIENT-SERVER APPLICATION TO DEMONSTRATE HTTP REQUEST AND RESPONSE MESSAGE EXCHANGE
DATE:	

AIM

To develop a Java program to implement a client-server application that demonstrates the exchange of HTTP request and response messages.

ALGORITHM

1. Server setup: Import java.net.* and java.io.* packages, then create a ServerSocket bound to a fixed port (e.g., 8080) and have it wait by calling accept(), which blocks until a client connects.
2. Client connection: Create a Socket on the client side that connects to the server's IP address and the same port number, establishing a TCP connection.
3. Client sends request: The client constructs an HTTP GET request string a request line (e.g., GET /index.html HTTP/1.1) followed by headers such as Host and Connection and writes it to the socket's output stream.
4. Server accepts and reads: Once accept() returns, the server obtains the input stream of the connected client socket, wraps it in a BufferedReader, and reads the incoming HTTP request line by line.
5. Server parses request: The server parses the request line to extract the HTTP method, the requested resource path, and the protocol version.
6. Server builds response: The server constructs an HTTP response consisting of a status line (e.g., HTTP/1.1 200 OK), response headers (Content-Type, Content-Length), and an HTML body.
7. Server sends response: The server obtains the output stream of the client socket and writes the complete response, then flushes the stream to ensure full transmission.
8. Client reads response: The client reads the response from its socket's input stream line by line, capturing the status line, headers, and body.
9. Display and close: The client displays the full HTTP response on the console; both the client and server then close their respective sockets to release the connection.
10. Server loop: The server loops back to step 1 (calling accept() again) to handle further incoming client requests, or terminates if shutdown is requested.

PROGRAM

HTTPServer.java

```
import java.io.*;
import java.net.*;

public class HTTPServer {
    public static void main(String[] args) {
        int port = 8080;
```

```

try (ServerSocket serverSocket = new ServerSocket(port)) {
    System.out.println("HTTP Server started on port " + port);

    while (true) {
        Socket clientSocket = serverSocket.accept();
        System.out.println("Client connected: " + clientSocket.getInetAddress());

        BufferedReader in = new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));
        PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

        String requestLine = in.readLine();
        System.out.println("Request received: " + requestLine);

        String line;
        while ((line = in.readLine()) != null && !line.isEmpty()) {
            System.out.println(line);
        }

        String body = "<html><body><h1>Hello from Java HTTP Server</h1></body></html>";
        out.println("HTTP/1.1 200 OK");
        out.println("Content-Type: text/html");
        out.println("Content-Length: " + body.length());
        out.println();
        out.println(body);
        out.flush();

        clientSocket.close();
        System.out.println("Response sent. Connection closed.\n");
    } catch (IOException e) {
        System.out.println("Server error: " + e.getMessage());
    }
}

HTTPClient.java
import java.io.*;
import java.net.*;

```

```

public class HTTPClient {
    public static void main(String[] args) {
        String serverAddress = "127.0.0.1";
        int port = 8080;

        try (Socket socket = new Socket(serverAddress, port)) {
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));

            out.println("GET /index.html HTTP/1.1");
            out.println("Host: " + serverAddress);
            out.println("Connection: close");
            out.println();
            out.flush();

            String responseLine;
            System.out.println("--- HTTP Response ---");
            while ((responseLine = in.readLine()) != null) {
                System.out.println(responseLine);
            }
        } catch (IOException e) {
            System.out.println("Client error: " + e.getMessage());
        }
    }
}

```

SAMPLE OUTPUT

```

Server side:
HTTP Server started on port 8080
Client connected: /127.0.0.1
Request received: GET /index.html HTTP/1.1
Host: 127.0.0.1
Connection: close
Response sent. Connection closed.

Client side:
--- HTTP Response ---
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 62

```

```
<html><body><h1>Hello from Java HTTP Server</h1></body></html>
```

PRE LAB QUESTIONS

1. What is HTTP, and which transport-layer protocol does it commonly run over?
2. What are the components of an HTTP request message (request line, headers, body)?
3. What are the components of an HTTP response message (status line, headers, body)?
4. What is the significance of the HTTP status code 200, and name two other common status codes.
5. What is the difference between persistent and non-persistent HTTP connections?

POST LAB QUESTIONS

1. How would you modify the server to handle different HTTP methods such as POST?
2. What would happen if the "Content-Length" header did not match the actual body length?
3. How can this simple server be extended to serve actual static files from disk?
4. Why is the "Connection: close" header included in the client's request?
5. How does this simplified server differ from a production-grade web server like Apache or Nginx?

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT

The Java program implementing a client-server application demonstrating HTTP request and response message exchange was developed and executed successfully

EX.NO.:12	STUDY OF NETWORK SIMULATION USING THE NS2/NS3 NETWORK SIMULATOR
DATE:	

AIM

To study network simulation concepts and simulate a simple wired network topology (nodes, links, and TCP/UDP traffic) using the NS2 (or NS3) Network Simulator, and to analyze the resulting trace/animation output.

ALGORITHM

1. Install and set up the NS2 simulator environment (or NS3, depending on availability) on a Linux system.
2. Open a text editor and create a new simulation script file with a `.tcl` extension (for NS2).
3. Initialize a new `Simulator` object and create trace and NAM (Network Animator) output files to log simulation events.
4. Define the required number of network nodes using the `\$ns node` command.
5. Create duplex links between the nodes, specifying bandwidth, delay, and queue type (e.g., DropTail).
6. Set up traffic agents: attach a TCP agent and sink (or UDP agent and a Null agent) to source and destination nodes respectively.
7. Attach traffic generators (e.g., FTP over TCP, or CBR over UDP) to the source agent to generate data traffic.
8. Schedule simulation events using `\$ns at <time> "<event>"`, such as starting traffic, and stopping the simulation after a defined duration.
9. Run the simulation using the `\$ns run` command, which generates trace (`.tr`) and animation (`.nam`) output files.
10. Open the `.nam` file using the Network Animator to visually observe packet flow, and analyze the `.tr` trace file for throughput, delay, and packet-loss statistics.

PROGRAM

(NS2 Tcl Script)

```
simple_topology.tcl
# Create a simulator object
set ns [new Simulator]

# Open trace and NAM files
set tracefile [open out.tr w]
$ns trace-all $tracefile
set namfile [open out.nam w]
```

```

$ns namtrace-all $namfile
# Create four nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
# Create links between nodes
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
# Set up a TCP connection between n0 and n3
set tcp [new Agent/TCP]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
$ns connect $tcp $sink
set ftp [new Application/FTP]
$ftp attach-agent $tcp
# Schedule events
$ns at 0.5 "$ftp start"
$ns at 4.5 "$ftp stop"
$ns at 5.0 "finish"
proc finish {} {
    global ns tracefile namfile
    $ns flush-trace
    close $tracefile
    close $namfile
    exec namout.nam&
    exit 0
}

# Run the simulation
$ns run

```

SAMPLE OUTPUT

Excerpt from out.tr trace file:

+ 0.5 0 2 tcp 1000 ----- 1 0.0 3.0 0 0

- 0.5 0 2 tcp 1000 ----- 1 0.0 3.0 0 0

r 0.510 0 2 tcp 1000 ----- 1 0.0 3.0 0 0

PRE LAB QUESTIONS

1. What is network simulation, and why is it useful before deploying real network infrastructure?
2. What is the difference between NS2 and NS3 in terms of architecture and scripting language?
3. What is the purpose of the Network Animator (NAM) in NS2?
4. What information is recorded in a trace (`.tr`) file generated by NS2?
5. What is a DropTail queue, and how does it relate to congestion in a simulated network?

POST LAB QUESTIONS

1. How would you calculate throughput and packet-loss rate from the generated trace file?
2. What changes would be needed to simulate a wireless network instead of a wired one in NS2?
3. Why might NS3 be preferred over NS2 for more modern or large-scale simulation studies?
4. How does changing the link bandwidth or delay affect the observed simulation results?
5. What are some limitations of discrete-event simulators like NS2/NS3 compared to real network testbeds?

MARK ALLOCATION

Details	Marks Allotted	Marks Awarded
Pre Lab Questions	10	
Algorithm	15	
Program Execution	30	
Post Lab Questions	10	
Lab Record	10	
Total	75	

RESULT

The network simulation was studied and a simple wired topology was simulated using the NS2 Network Simulator. The trace file and NAM animation successfully demonstrated packet flow, TCP connection setup, and data transfer between simulated nodes.